

Views em SQLite

Simplificando consultas e organizando seu banco de dados

Uma abordagem prática para desenvolvedores e DBAs que buscam clareza, reutilização e eficiência no dia a dia.



Conceito Fundamental

O que é uma View?

Uma **View** (ou visão) é uma **tabela virtual** criada a partir de uma instrução `SELECT`. Ela não armazena dados fisicamente no disco — em vez disso, guarda apenas a **definição da consulta**, que é executada dinamicamente toda vez que a view é acessada.

Pense na view como um **atalho nomeado** para uma consulta SQL que você usaria repetidamente. Do ponto de vista do desenvolvedor, ela se comporta exatamente como uma tabela comum: você pode fazer `SELECT * FROM nome_da_view` e obter os dados esperados.

- ❏ Em SQLite, views são criadas com `CREATE VIEW` e existem enquanto o banco de dados existir.

View vs. Tabela Real

- Tabela real → armazena dados em disco
- View → armazena apenas o `SELECT`
- Tabela real → ocupa espaço crescente
- View → zero dados redundantes

Principais Objetivos das Views

As views são construídas sobre quatro pilares fundamentais que justificam seu uso em projetos de qualquer tamanho.



Simplificar Consultas

Encapsula JOINS complexos e subqueries em uma interface simples e reutilizável.



Reutilizar Código (DRY)

Escreva a lógica uma vez. Consulte de qualquer lugar. Evite duplicação de código SQL.



Segurança e Abstração

Expõe apenas as colunas necessárias, ocultando estrutura interna e dados sensíveis.



Facilidade de Manutenção

Altere a view em um único lugar e todas as consultas dependentes se atualizam automaticamente.

Pilar 1

Simplificando JOINS Complexos

Sem uma view, cada desenvolvedor precisa escrever — e memorizar — toda a estrutura de JOIN toda vez. Com uma view, a complexidade desaparece para o consumidor da consulta.

❌ Sem View — consulta repetida

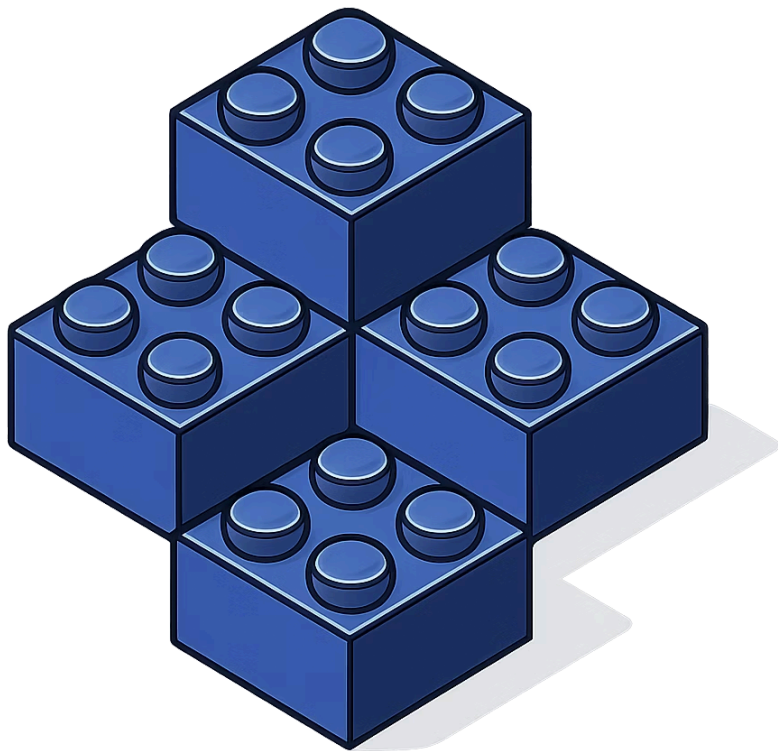
```
SELECT
  p.nome AS produto,
  c.nome AS categoria,
  f.nome AS fornecedor,
  p.preco,
  p.estoque
FROM produtos p
INNER JOIN categorias c
  ON p.categoria_id = c.id
INNER JOIN fornecedores f
  ON p.fornecedor_id = f.id
WHERE p.estoque > 0;
```

✅ Com View — simples e limpo

```
-- Criação (feita uma vez)
CREATE VIEW vw_produtos_completo AS
SELECT
  p.nome AS produto,
  c.nome AS categoria,
  f.nome AS fornecedor,
  p.preco,
  p.estoque
FROM produtos p
INNER JOIN categorias c ON p.categoria_id = c.id
INNER JOIN fornecedores f ON p.fornecedor_id = f.id;

-- Uso (feito sempre que necessário)
SELECT *
FROM vw_produtos_completo
WHERE estoque > 0;
```

Reutilização de Código — Princípio DRY



O princípio **DRY** (*Don't Repeat Yourself*) é um dos pilares da engenharia de software de qualidade. Em SQL, violamos o DRY toda vez que copiamos e colamos a mesma consulta em múltiplos lugares — relatórios, telas, exportações.

Com views, você centraliza a lógica de negócio em um único artefato no banco de dados. Qualquer sistema, linguagem ou ferramenta que acesse o banco pode usar a mesma view, garantindo **consistência de dados** e eliminando a manutenção distribuída.

- ❑ Uma alteração na regra de negócio impacta apenas a definição da view, não todos os pontos de uso.

Pilar 3

Segurança e Abstração de Dados

Views permitem **expor apenas o que é necessário**, ocultando colunas sensíveis e a estrutura interna do banco de dados — uma prática essencial em sistemas multi-usuário.

Ocultação de Colunas Sensíveis

Crie uma view de `clientes` que exclui `senha_hash`, `cpf` e `dados_cartao`. O sistema de relatórios acessa a view sem nunca ver esses campos.

Abstração da Estrutura do Banco

Se você renomear uma tabela ou reorganizar colunas, basta atualizar a view. As aplicações que a consomem não precisam ser alteradas — a interface permanece estável.

Filtro de Linhas por Contexto

Views podem incluir cláusulas `WHERE` fixas, garantindo que certos usuários ou módulos vejam apenas os registros pertinentes ao seu escopo de acesso.

Criando e Gerenciando Views no SQLite

Criar uma View

```
CREATE VIEW nome_da_view AS  
SELECT coluna1, coluna2, ...  
FROM tabela  
WHERE condicao;
```

Consultar uma View

```
-- Exatamente como uma tabela  
SELECT * FROM nome_da_view;  
  
SELECT coluna1  
FROM nome_da_view  
WHERE coluna2 = 'valor'  
ORDER BY coluna1;
```

Listar todas as Views do banco

```
SELECT name, sql  
FROM sqlite_master  
WHERE type = 'view'  
ORDER BY name;
```

Remover uma View

```
DROP VIEW IF EXISTS nome_da_view;
```

 O `IF EXISTS` evita erros caso a view não exista. Use sempre para scripts que possam ser executados múltiplas vezes.



Casos de Uso Reais

Sistema de Gestão de Oficina e Assistência Técnica

Veja como views resolvem problemas concretos de negócio em um sistema real, tornando relatórios e alertas operacionais muito mais simples de implementar e manter.

Caso de Uso

View de Estoque Baixo

Um dos alertas mais críticos em uma oficina: identificar rapidamente quais peças estão abaixo do nível mínimo de reposição para evitar paralisação de serviços.

```
CREATE VIEW vw_estoque_baixo AS
SELECT
  p.id,
  p.nome      AS peca,
  p.quantidade AS qtd_atual,
  p.estoque_min AS qtd_minima,
  (p.estoque_min - p.quantidade) AS qtd_repor,
  f.nome      AS fornecedor
FROM pecas p
INNER JOIN fornecedores f
  ON p.fornecedor_id = f.id
WHERE p.quantidade <= p.estoque_min
ORDER BY qtd_repor DESC;

-- Uso diário pelo almoxarife:
SELECT * FROM vw_estoque_baixo;
```

O que a view resolve

- Elimina JOIN manual em cada relatório
- Calcula automaticamente a quantidade a repor
- Inclui o contato do fornecedor direto no resultado
- Ordenada por prioridade de reposição

✅ O almoxarife só precisa executar um `SELECT *` para ver tudo que precisa agir.

Limitações das Views no SQLite

Entender as limitações é tão importante quanto conhecer os benefícios. Views no SQLite têm restrições que devem guiar seu uso correto.

1

Read-Only por Padrão

Views no SQLite são, por padrão, apenas para leitura. Tentativas de `INSERT`, `UPDATE` ou `DELETE` diretamente em uma view resultam em erro.

2

Processamento em Tempo Real

A cada acesso, a query subjacente é executada do zero. Views com `JOINS` pesados ou grandes volumes de dados podem impactar a performance.

Boas Práticas ao Usar Views

1 Adote uma convenção de nomenclatura clara

Use prefixos como `vw_` para diferenciar views de tabelas reais. Ex: `vw_estoque_baixo`, `vw_relatorio_mensal`. Isso facilita a leitura de queries e a manutenção por outros desenvolvedores.

2 Documente o propósito de cada view

Mantenha um arquivo SQL de criação com comentários explicando o objetivo da view, quais tabelas ela usa e quem a consome. Views sem documentação se tornam um mistério após alguns meses.

3 Evite views sobre views

Criar views que consultam outras views pode gerar cascatas de performance difíceis de depurar. Prefira sempre consultar as tabelas base diretamente na definição da view.

Resumo Final

O que Levar para o Dia a Dia

Views são atalhos inteligentes

Não armazenam dados, apenas a definição do `SELECT`. Simples de criar, poderosas em uso.

DRY na prática

Escreva a lógica uma vez no banco e reutilize em qualquer ponto do sistema — API, relatório, ou BI.

Segurança sem custo extra

Exponha apenas o que precisa ser exposto. Oculte estrutura interna e dados sensíveis com elegância.

Atenção às limitações

Read-only por padrão. Monitore performance e documente cada view criada.

"Uma boa view é como uma boa API: esconde a complexidade e entrega exatamente o que o consumidor precisa."